

Climate.xyz Transformer

Cherif Benham, Madison Davis

August 21, 2023

1 Abstract

The task at hand was to produce a transformer encoder-decoder model that can produce accurate information for consumers and third-party company investors concerning sustainable development, including labor, equipment, and energy production. The model should be able to handle multiple forms of data, including unstructured text files, structured query data, and off-file internet information. The resulting model is able to perform on topics relating to real-life solar, vehicle, and house equipment products, workers, and company data. This paper discusses documentation relating to transformer architecture, current model setup, data preparation, and exemplar queries.

2 Transformer Architecture

2.1 Input Vectors

A transformer model is a transduction model, which means it encounters both training and testing datasets during its training. A single datapoint for a transformer model can be an incomplete sentence in which the model has to determine the most probable next word in the sequence. Upon inputting the sentence, each word or token in the sentence is converted into a vector representation. This vector representation is called a token embedding. As an example, the word "cat" may become the token embedding vector $[0.12, 0.24]$. The length of this token embedding vector is fixed and is typically determined by the developer when designing the model. The conversion from the word to the token embedding can be achieved either using pre-trained word embeddings, such as Word2Vec, GloVe, or FastText, or by learning token embeddings from scratch during the model training. These embeddings capture the semantic meaning of individual tokens and are essential for the model to understand the input's content. Each resulting vector within the sentence is now called a token embedding.

2.2 Position Vectors

Since the transformer doesn't inherently understand the sequential order of the tokens, positional embeddings are introduced to provide the model with

information about the positions of words in the sentence. Each word has its own position embedding vector that is element-wise added to the token embedding vector. An example position embedding vector could be $[0.2, 0.5]$. Each number in the position embedding vector comes from using sine and/or cosine functions with varying frequencies. The numbers are usually random at first and then learned through training. The individual numbers don't have direct, intuitive meanings. Rather, the entire vector provides a unique encoding for each position within the sequence. A positional embedding vector is element-wise added to the token embedding vector to create our now up-to-date vector.

2.3 Encoder

The finalized set of vectors are now placed through an architecture called the encoder. The encoder allow us to capture complex relationships within our vectors, mainly what words are most important to understand. The encoder is performed multiple times simultaneously on the same group of vectors for several reasons:

1. Expressiveness: a single layer might be limited in how many relationships it can capture across the sentence.
2. Feature Refinement: each layer refines the features extracted by the previous layer, thus making the model more accurate.

This multi-layer action is symbolized by visually stacking the singular encoder architecture one on top of the other. Each encoder layer that performs its own separate calculations is referred to as an "attention head." A single encoder is made up of two blocks: a multi-head attention part and a feed forward part.

For a singular layer, in the multi-head attention part, apply bias operations and matrix multiplication to each token vector to obtain its query, key, and value vectors. As an example, the Query Vector for a specific token vector can be obtained by doing $\text{Query} = \text{QueryLinear}(\text{PositionEmbedding} + \text{InputEmbedding})$. Next, for each token query vector, take the dot product between that and all other tokens' key vectors. Divide this result by the square root of the dimension of the key vector and apply a softmax function to obtain a final weight number. At the end of calculating this attention head, you'll have a vector where the i th number represents the relevance of the i th word in the sentence.

Once we've done this several times across multiple attention heads, concatenate all the attention heads together to create one large vector that gives plenty of pattern-recognizing, relevant information about the sentence.

2.4 Feedforward Neural Network

The next step after the encoder is going through the feedforward neural network. A feedforward neural network block is incorporated to allow the model

to adaptively adjust and refine the representations. It can learn to emphasize certain features, suppress noise, and create more abstract and higher-level representations. This neural network applies a linear transformation, then a Rectified Linear Unit (ReLU) activation, and finally a another linear transformation on the vector.

2.5 Decoder

After the encoder comes the decoder architecture. The decoder generates output based on what information was learned from the encoder. When training the model, a desired output word is typically inserted into the decoder as a way for the transformer to check that what it outputs is proficient. This desired output word first goes through a multi-head attention layer to turn it into a vector. Next, the encoder result goes through another encoder-like block, but now the desired output word vector is added to the feedforward neural network.

If the model is not being trained and a sentence is simply being inserted to get an output word, then encoder result simply goes through another encoder-like block, without feedback from a desired output word.

The output from the decoder is a vector that captures the model's prediction for the next token in the output sequence. This vector is called the "logits vector" or "output distribution." It represents the model's confidence scores or probabilities for each possible token in the transformer's vocabulary to be the next token in the generated sequence. The vocabulary list of words used in transformer models, including both the encoder and decoder, comes from the training data used to train the model. In the context of natural language processing, the vocabulary consists of all the unique words or subword units that appear in the training corpus. Once we have this vector, simply choose the highest-scoring prediction using a linear transformation so as to get the predicted word.

3 The Rise of Transformers

To understand why data analysts felt the need to introduce transformer architecture, one must look at what was previously being used to develop sentences: Recurrent Neural Networks (RNNs). RNNs generate a sequence of hidden states based on the previous hidden state and current input. The sequential nature of RNNs does not allow for parallelization, increases memory usage, and poses challenges with large data. Transformers are better than RNNs because they require fewer sequential operations ($O(1)$) compared to RNNs ($O(n)$). Moreover, transformers have shorter paths between any combination of inputs, allowing for more effective learning.

4 Data Preparation

4.1 Structured Data

Dataset Name	Number Entries	Attributes
SolarPanelInstallers	155	Company, Company Address, Phone ,Website, Headquarters, Installation Type, Grid Type
SolarPanelWorkers	155	SIRET Number, company, address, postal code, commune, latitude, longitude, telephone, email, website URL, qualification code, certification name, certification distributor, certification description, start date and end date for their certificate, certificate URL, specific work description, work domain, work title, if they can work as an individual, work location, opening hours, number of reviews, rating
SolarPanelProducts	280	product name, product image, product URL, number of reviews, product price, delivery details, number of product sub-models
HouseEquipment	2688	equipment name, price per unit, equipment attributes, equipment description, availability, brand, brand URL, media, origin, rating, number of reviews, seller, seller URL, SKU, product URL
VehicleModels	3561	brand name, model type, model name, number of car seats, trunk volume, wheel base, empty weight, tank volume, urban consumption, mixed consumption, extra urban consumption, CO2 emissions, number of cylinders, valves per cylinder, displacement, din power, engine torque, fiscal power, engine position, power supply, max speed, supercharger method, transmission type, box type, number of speeds, distribution, bodywork, front tire width, rear tire width, front tire high/low ratio, rear tire high/low ratio, front suspension, rear suspension, front rims diameter, rear rims diameter, maintenance interval, month guaranteed, manufacturer's country of origin, start and end of marketing, vehicle image URL, length, width, height, website URL, year made

4.2 Unstructured Data

Source	Topic	Report Title	Year
IEA	Solar	National Survey Report of PV Power Applications in France	2019
IEA	Solar	Technology Roadmap of Solar Photovoltaic Energy	2010
IEA	Solar	Designing New Materials for Photovoltaics	2021
IEA	Solar	France 2021 Energy Policy Review	2021
IEA	Solar	2050 Conditions and Requirements	2021
IRENA	Solar	End-Of-Life Management of Solar Panels	2016
IRENA	Solar	Future of Solar Photovoltaic	2019
RTE	Solar	Green Financing Framework	2021
BEUC	Solar	Affordable Green Loans for Consumers	2021
FrenchGov	Solar	Solar: uses, tariffs, qualifications, aid, objectives	2023
Arxiv	Solar	Solar Panel Potential for Building Energy	2018
ScienceDirect	Solar	Review of the performance of residential PV systems in France	2011
ITF	Vehicle	Understanding Consumer Vehicle Choice A New Car Fleet Model	2019
ITF	Vehicle	Policy Priorities for Decarbonising Urban Passenger Transport	2018
IEA	Vehicle	France 2021 Energy Policy Review	2021
IEA	Vehicle	Global Electric Vehicle Outlook 2022	2022
FrenchGov	Vehicle	Green Bonus for the Acquisition of Low-Emission Vehicles	2023
FrenchGov	Vehicle	Overview of government policies to promote ultra-low emission vehicles	2018
FrenchGov	Vehicle	French 2030 Plan	2021
FrenchGov	Vehicle	Promulgation of the Climate and Resilience law	2021
FrenchGov	Vehicle	Heavy vehicles using clean energy, Ecological and Transition	2021
FrenchGov	Vehicle	Developing clean automobiles and electric cars	2023
FrenchGov	Vehicle	Multi-year energy programming	2020
FrenchGov	Vehicle	State Investments in Transport	2019
FrenchGov	Vehicle	French National Roadmap for Hydrogen	2019
Arxiv	Vehicle	Biggest Vehicle Polluters and Emissions Reduction	2021
Arxiv	Vehicle	Reduce Energy Usage for Public Mixed-Fleet Vehicles	2020
Arxiv	Vehicle	Electric vs Conventional Fuel Cars	2021
Arxiv	Vehicle	Effects of Alternate Fuel for Fleet CO2 Emissions	2021
Deloitte	Vehicle	Fleet Management in Europe	2017

The aforementioned data was processed in a way that would benefit the user most in terms of simplicity of use and time complexity. The optimal transformer uses both unstructured and structured data that was either previously loaded into the model or would be reachable through internet API search queries. Due to the overload of data, the model faced difficulty in exceeding maximum token limits. This was especially true when the number of structured datasets were increased in comparison to previously-loaded Langchain tools or unstructured file text. To combat this issue while maintaining all data within a singular architecture, the user's query would be utilized in semantic search to determine what relevant structured datasets were to be loaded as tools in that instance. The

following code snippet demonstrates how the most similar structured database was selected off of a user's query. The csv files refers to a folder containing all potential .csv files for query extraction. The description files refers to a singular .xlsx document that details each csv's name and content.

```
def find_most_similar_db(query):
    csv_files = f"..."
    csv_paths = glob.glob(csv_files)
    description_files = f"..."
    descriptions = glob.glob(description_files)
    first_sheet = pd.read_excel(descriptions[0])
    db_names = first_sheet.iloc[:,0].tolist()
    db_headers = first_sheet.iloc[:,1].tolist()

    texts = []
    for i in range(0,
len(first_sheet.iloc[:,0].tolist())):
        texts.append(db_names[i] + ": "
+ db_headers[i])
    embedding_function =
SentenceTransformerEmbeddings(model_name =
"all-MiniLM-L6-v2")

    ids = [name for name in db_names]
    db = Chroma.from_texts(texts, embedding_function,
ids = ids, collection_name="csvs",)
    ids = db.get()["ids"]
    db.delete(ids =ids)
    db.persist()
    db = Chroma.from_texts(texts, embedding_function,
ids = ids, collection_name="csvs",)

    best_match =
db.similarity_search(query = query,
distance_metric = "cos", k = 1)

    for name in db_names:
        if name in best_match[0].page_content:
            for file in csv_paths:
                if name in file:
                    description = db_headers[db_names.index(name)]
                    return (file, name, description)
```

5 Current Model Structure

The current model is designed as an OpenAI large language model transformer with inherited memory and a React Description Agent. The agent decides what specific tool would be best used to answer the question at hand. The first tool is a list of all unstructured, PDF-formatted, previously-loaded documents. This was done by using The second tool is an open Internet search. The final is a singular structured query document whose selection is based on the user's query.

```
from langchain import OpenAI, LLMMathChain
from langchain.agents import
create_pandas_dataframe_agent, initialize_agent

def perform(message, history):
    llm = OpenAI(temperature = 0)
    memory = ConversationBufferMemory
    (memory_key="chat_history")

    file_name, name, description =
    find_most_similar_db(message)
    db = create_csv_tools(file_name, name, description)
    db_tool = create_pandas_dataframe_agent
    (OpenAI(temperature=0), db, verbose=True)
    pdf_tool = RetrievalQA.from_chain_type
    (llm=llm, chain_type='map_reduce',
    retriever=pdf_db.as_retriever(search_type=
    "similarity", search_kwargs={"k":1}),
    return_source_documents=True, verbose=True,
    input_key="question")

    tools = [
        Tool(
            name="PDF Search Queries",
            func=lambda message: pdf_tool(
                {"question": message}),
            description="useful for when
            you need to answer questions.
            Input should be a fully formed question."
        ),
        Tool(
```

```

        name="Internet Search Queries",
        func = DuckDuckGoSearchRun(),
        description = "Useful for when you
        want to search the internet for answers."
    ),
    Tool(
        name = "Database Search Queries",
        func = db_tool.run,
        description = "Useful for searching up
        numerical data or specific products,
        installers , and brands/companies"
    )
]

agent = initialize_agent(
    tools ,
    llm ,
    agent=
    AgentType.CONVERSATIONAL_REACT_DESCRIPTION,
    memory=memory,
    verbose=True,
)
output = agent.run(input=message)
return output

```

The current model is designed as an OpenAI large language model transformer with inherited memory and a React Description Agent. The agent decides what specific tool would be best used to answer the question at hand. The first tool is a list of all unstructured, PDF-formatted, previously-loaded documents. This was done by using The second tool is an open Internet search. The final is a singular structured query document whose selection is based on the user's query.

6 Model Testing

The model's structure was constructed in a manner that could be integrated well with web applications. Climate.xyz's transformer model was tested in a Flask-Gradio ML environment. The following queries were constructed to test the accuracy of the model's answer and the depth of content with which it responded.

Tool Targeted	Goal	Question	Answer
Tool 1: Vehicles	Find Attribute	Search your pdfs and tell me about the The International Transport Forum	The International Transport Forum (ITF) is an intergovernmental organization with 59 member countries that works to foster a deeper understanding of the role of transport in economic growth, environmental sustainability, and social inclusion. It produces the Case-Specific Policy Analysis series, which presents topical studies on specific issues carried out by the ITF in agreement with local institutions.
Tool 1: Solar Panel Installers	Find Attribute	What is 1001 Soleils's company address?	The top 5 solar panel installers are 1001 Soleils, 123 Photovoltaïque, 2 APC, 21 Panneaux Solaires, and 2B Energy Sarl
Tool 2: Internet	Find Attribute	Search the internet and tell me about the The International Transport Forum.	The International Transport Forum is an intergovernmental organization within the OECD system that acts as a think tank for transport policy issues and organizes the annual global summit of transport ministers. It is the only global body with a mandate for all modes of transport and is led by Secretary-General Young Tae Kim.
Tool 3: Vehicles	Find Attribute	Search your database and tell me some vehicle brand names.	Here are some vehicle brand names: Renault and Peugeot.
Tool 3: Vehicles	Find Attribute: Conditional	Give me a vehicle model name that has a tank of 45 and an urban consumption of 3.8 1.5 DCI 105 PRIVILEGE.	The vehicle model name that has a tank of 45 and an urban consumption of 3.8 is "IV (2) 1.5 DCI 110 ENERGY STEEL".
Tool 3: Vehicles	Math: Count	How many total vehicle models are there in the database?	There are a total of 3561 vehicle models in the database.
Tool 3: Vehicles	Math: Average	Give me the average number of seats in a vehicle.	The average number of seats in a vehicle is approximately 4.86
Tool 3: Vehicles	Math: Max	Give me the highest CO2 vehicle emissions in your database.	The highest CO2 vehicle emissions in the database is 204.0.
Tool 3: Vehicles	Math: Min	Give me the smallest urban consumption for a vehicle in your database.	The smallest urban consumption for a vehicle in the dataframe is 0.0 L / 100 km.